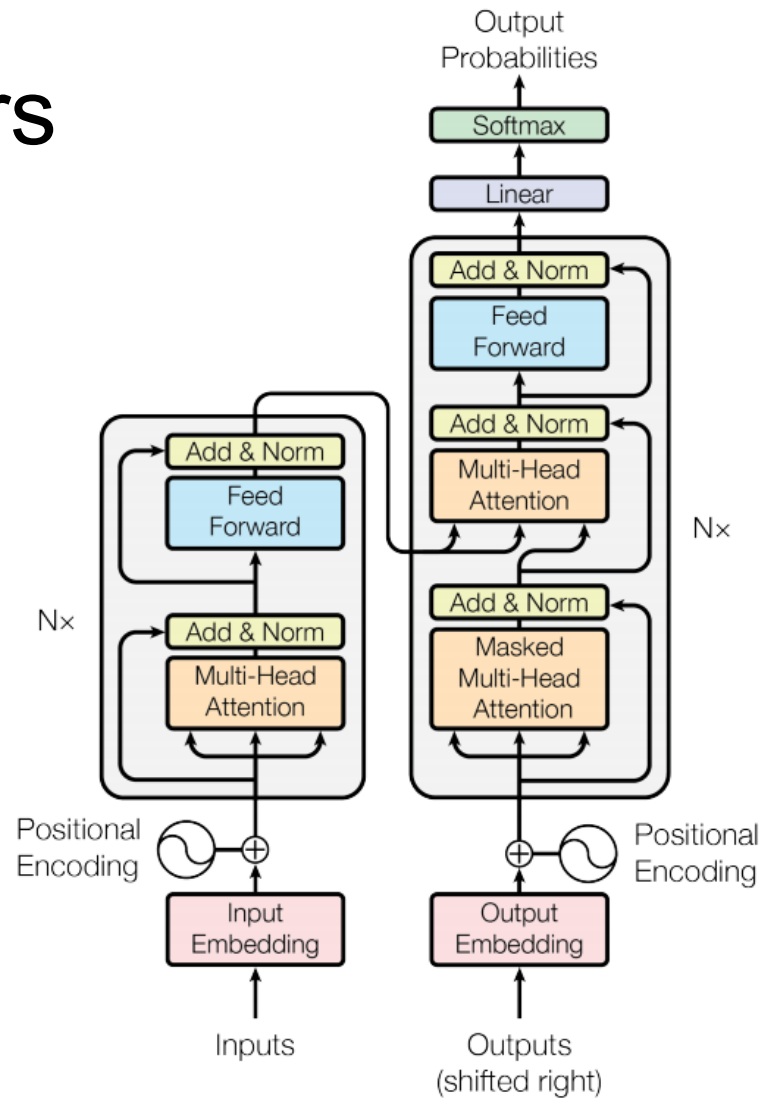


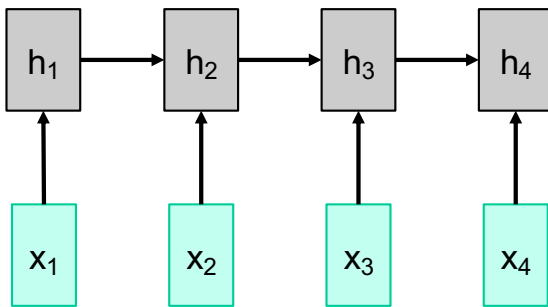
Transformers



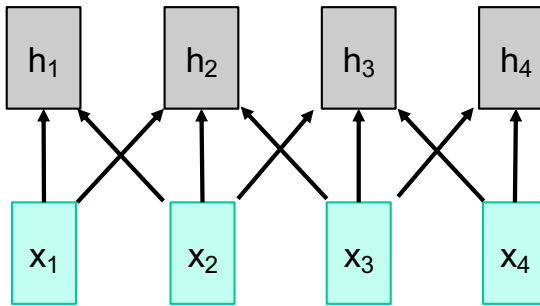
Many slides adapted from [J. Johnson](#)

Sequence modeling beyond RNNs

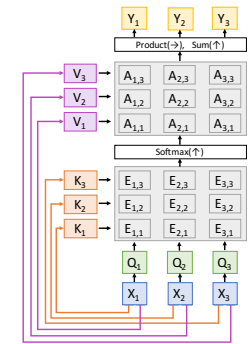
RNNs



1D convolutional networks



Transformers



Works on **ordered sequences**

- Pros: Not limited by fixed context size (in principle): After one RNN layer, h_T "sees" the whole sequence
- Con: Hidden states have limited expressive capacity
- Con: Not parallelizable: need to compute hidden states sequentially

Works on **multidimensional grids**

- Pro: Each output can be computed in parallel (at training time)
- Con: Bad at long sequences: Need to stack many conv layers for outputs to "see" the whole sequence

• Works on **sets of vectors**

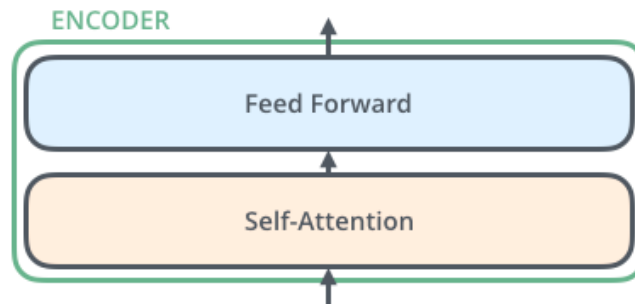
Outline

- Key-value-query attention
- Transformer attention models
- Other details of transformer architecture
- Large language models: the early years

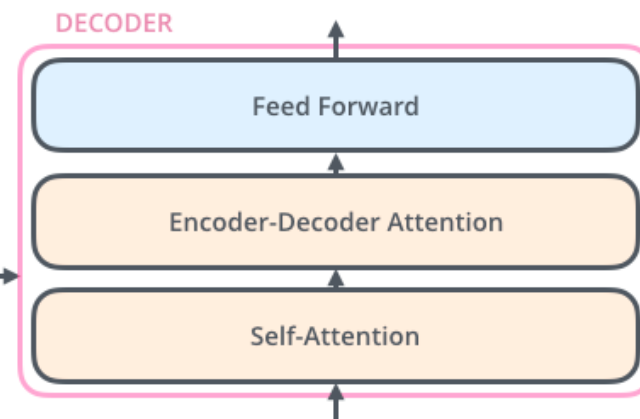
Basic transformer model

- Sequence-to-sequence architecture using *only point-wise processing and attention* – no recurrent units or convolutions

Encoder: receives entire input sequence and outputs encoded sequence of the same length



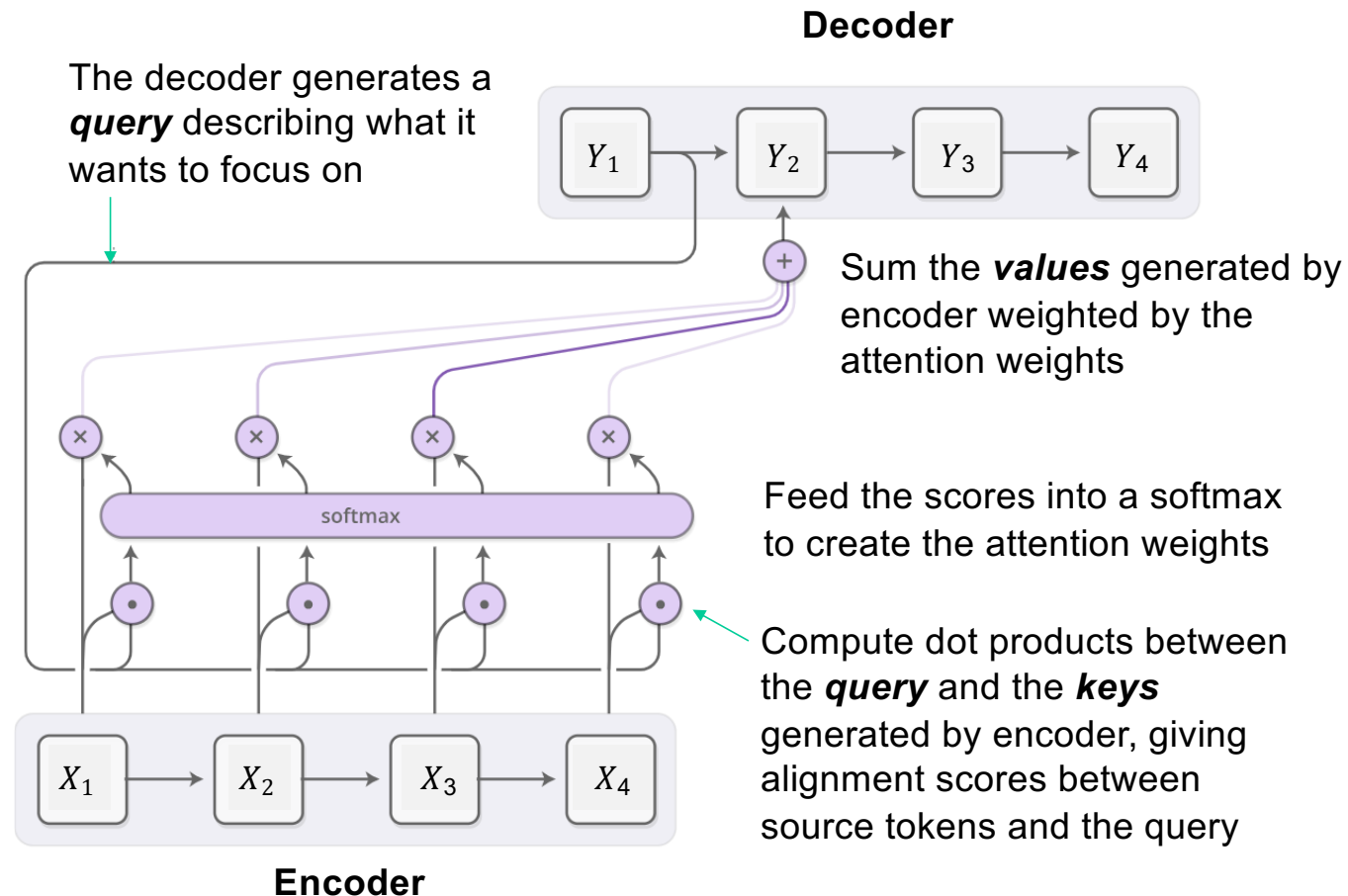
Decoder: predicts next token conditioned on encoder output and previously predicted tokens



A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, I. Polosukhin, [Attention is all you need](#), NeurIPS 2017

[Image source](#)

Key-Value-Query attention model



Key-Value-Query attention model

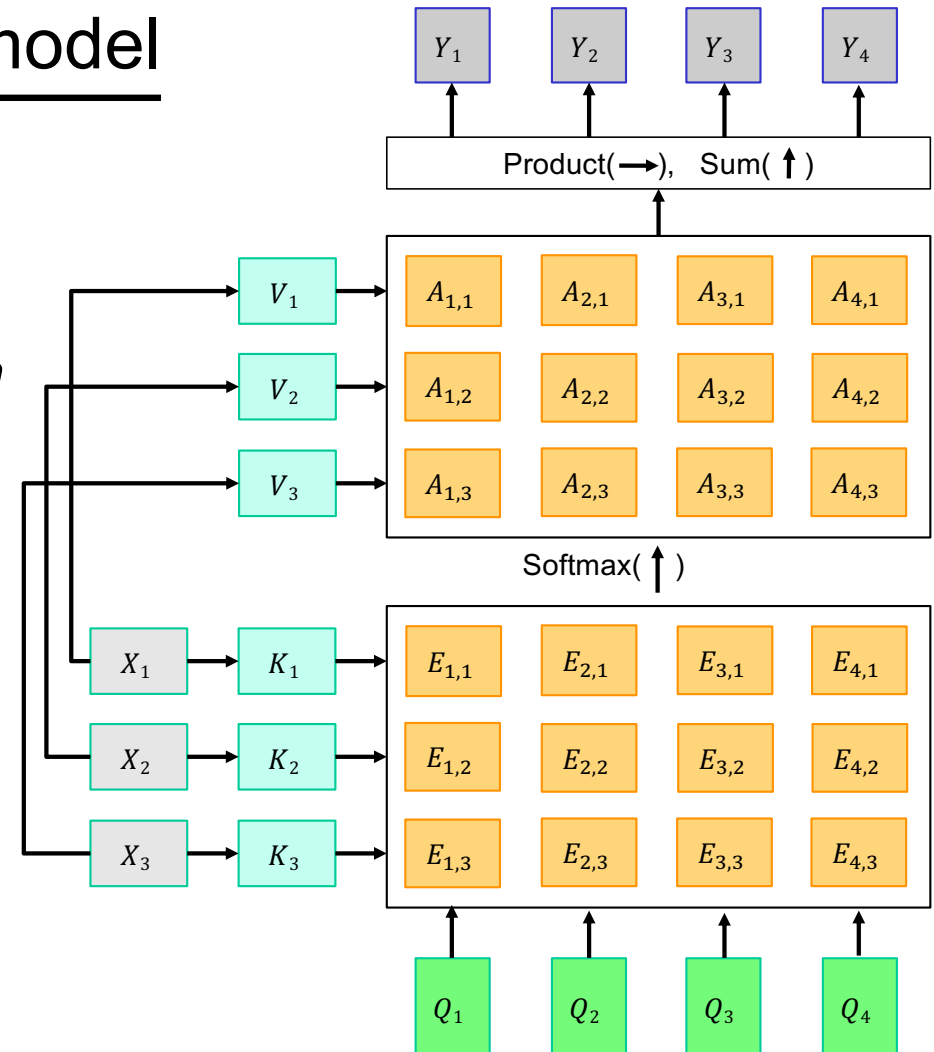
- Key vectors: $K = XW_K$
- Value Vectors: $V = XW_V$
- Query vectors
- Similarities: *scaled dot-product attention*

$$E_{i,j} = \frac{(Q_i \cdot K_j)}{\sqrt{D}} \text{ or } E = QK^T / \sqrt{D}$$

(D is the dimensionality of the keys)

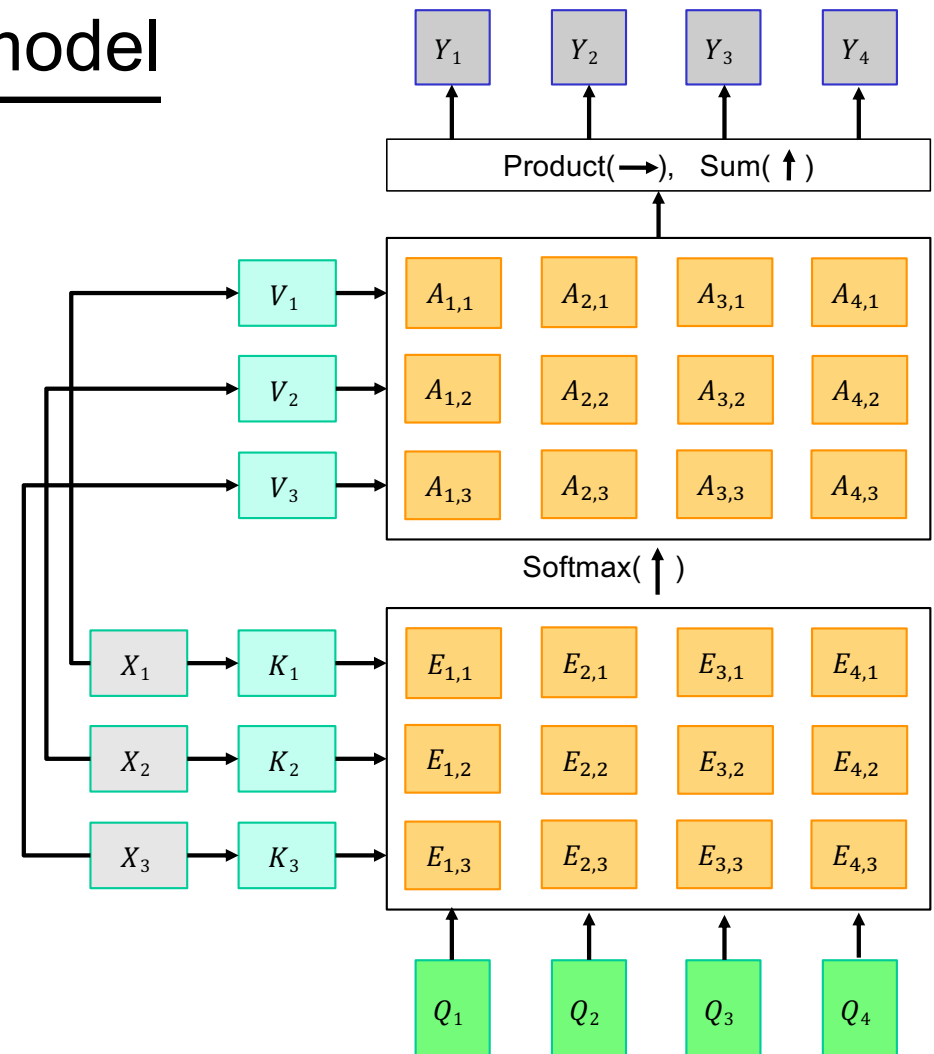
- Attn. weights: $A = \text{softmax}(E, \text{dim} = 1)$
- Output vectors:

$$Y_i = \sum_j A_{i,j} V_j \text{ or } Y = AV$$

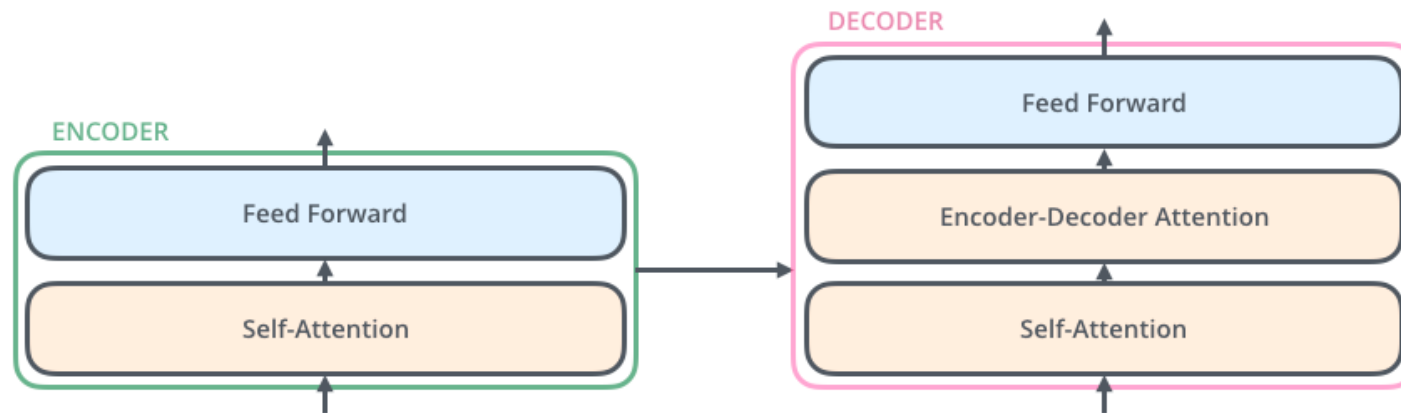


Key-Value-Query attention model

- How does permuting the order of the *queries* change the output?
- How does changing the order of the *keys/values* change the output?



Attention mechanisms



- **Encoder self-attention:** queries, keys, and values come from previous layer of encoder
- **Decoder self-attention:** values corresponding to future decoder outputs are masked out
- **Encoder-decoder attention:** queries come from previous decoder layer, keys and values come from output of encoder

Self-attention

- Used to capture context *within the sequence*

The animal didn't cross the street because **it** was too tired .

As we are encoding “it”, we should focus on “the animal”

The animal didn't cross the street because **it** was too wide .

As we are encoding “it”, we should focus on “the street”

[Image source](#)

Self-attention layer

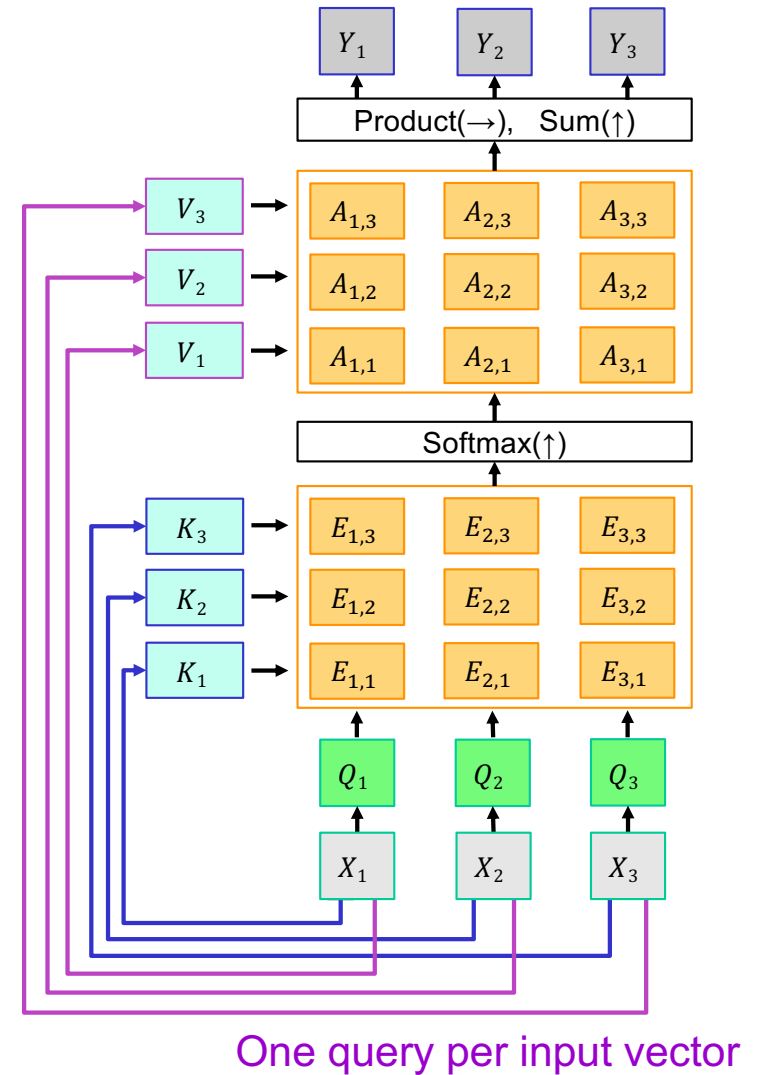
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$
- Value vectors: $V = XW_V$
- Similarities: *scaled dot-product attention*

$$E_{i,j} = \frac{(Q_i \cdot K_j)}{\sqrt{D}} \text{ or } E = QK^T / \sqrt{D}$$

(D is the dimensionality of the keys)

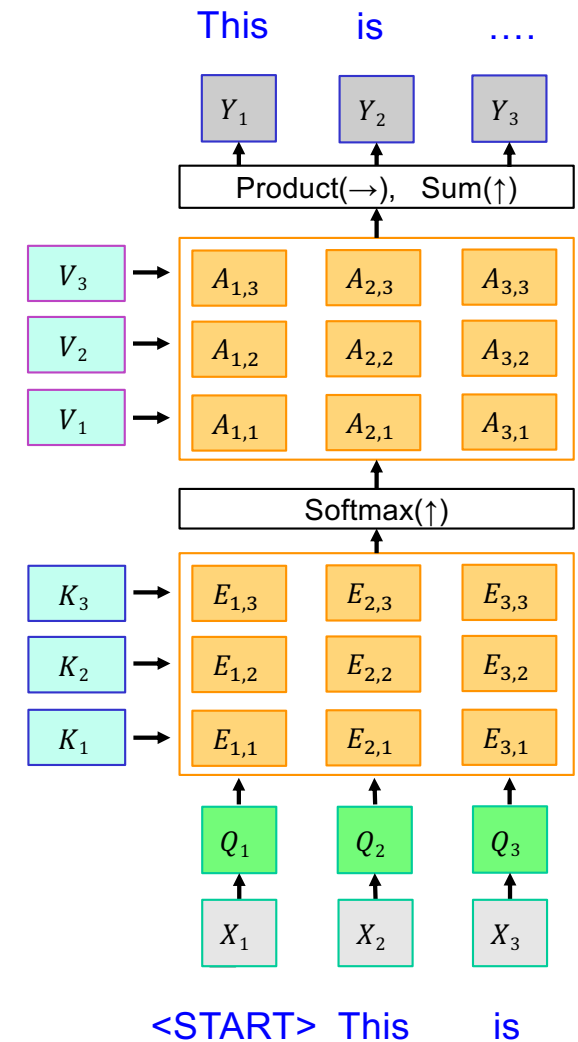
- Attn. weights: $A = \text{softmax}(E, \text{dim} = 1)$
- Output vectors:

$$Y_i = \sum_j A_{i,j} V_j \text{ or } Y = AV$$



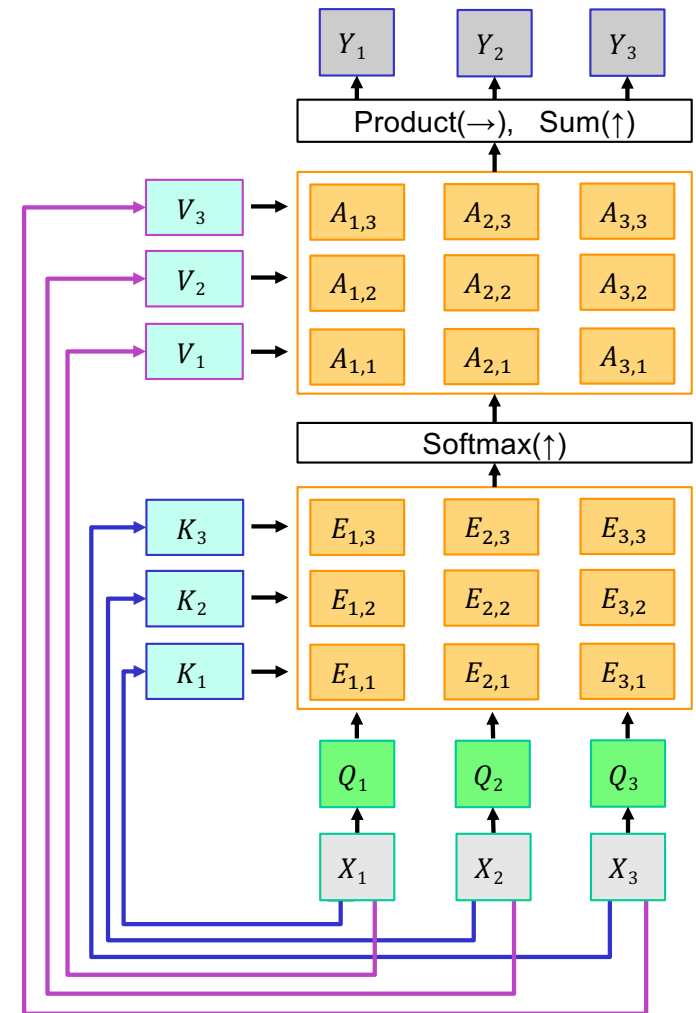
Masked self-attention layer

- The decoder should not “look ahead” in the output sequence



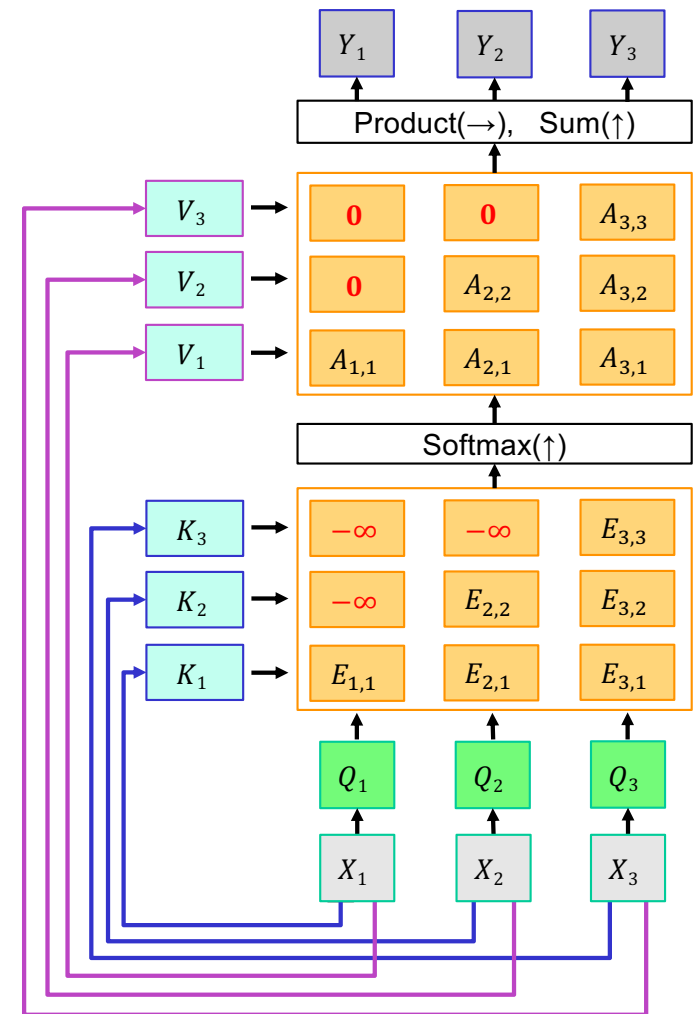
Masked self-attention layer

- The decoder should not “look ahead” in the output sequence

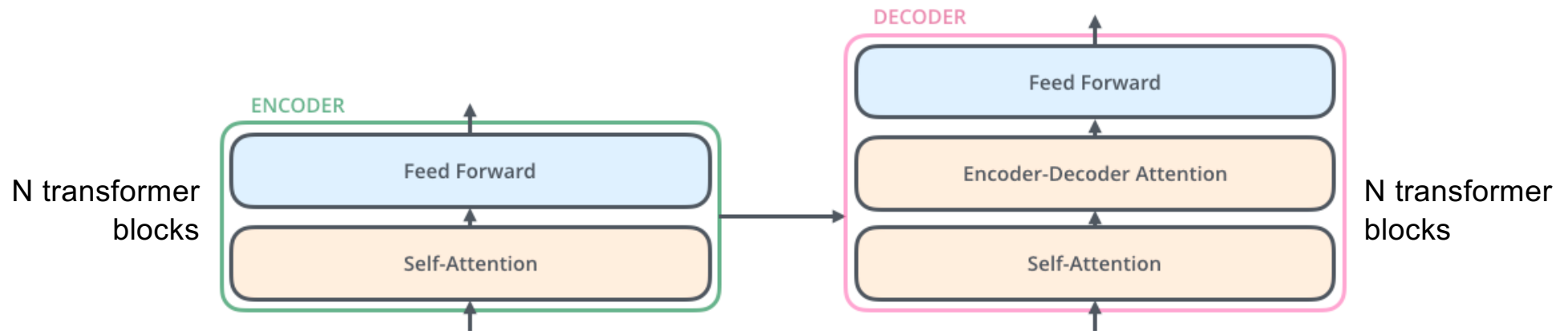


Masked self-attention layer

- The decoder should not “look ahead” in the output sequence



Attention mechanisms: Summary



- **Encoder self-attention:** queries, keys, and values come from previous layer of encoder
- **Decoder self-attention:** values corresponding to future decoder outputs are masked out
- **Encoder-decoder attention:** queries come from previous decoder layer, keys and values come from output of encoder

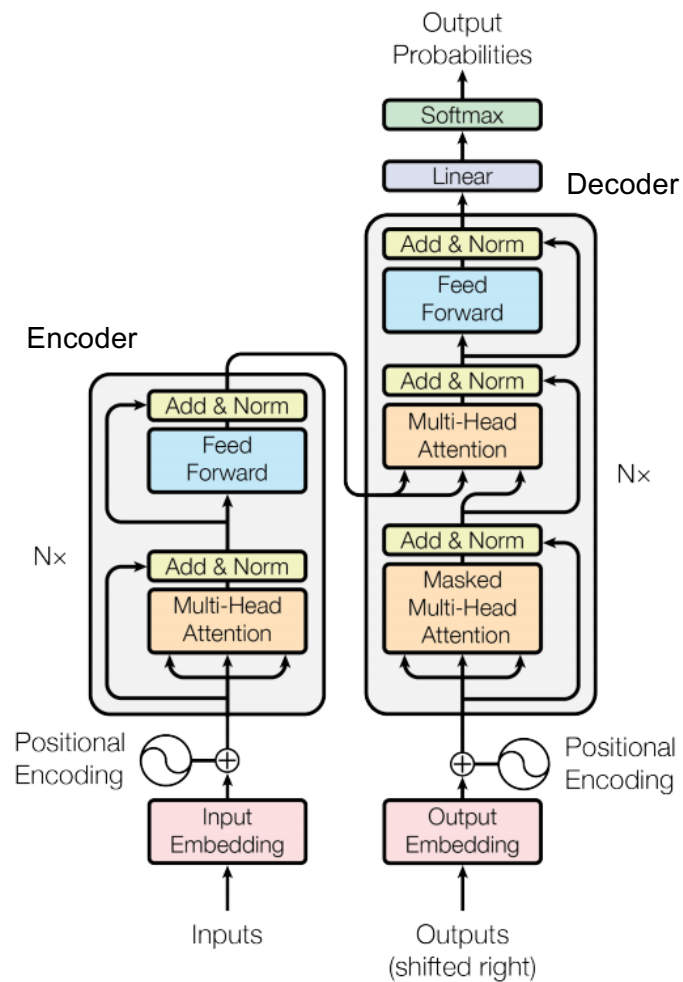
Attention mechanisms: Illustration

<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

Outline

- Key-value-query attention
- Transformer attention models
- Other details of transformer architecture

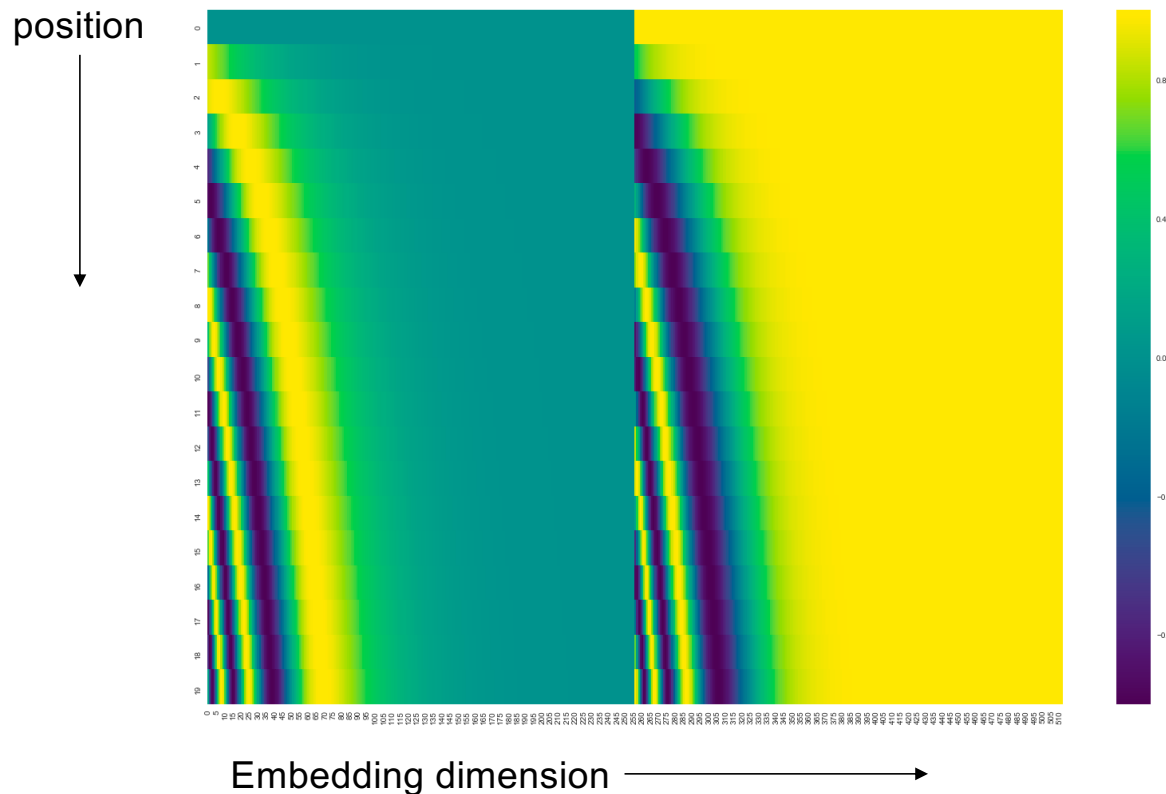
Transformer architecture: Details



A. Vaswani et al., [Attention is all you need](#), NeurIPS 2017

Positional encoding

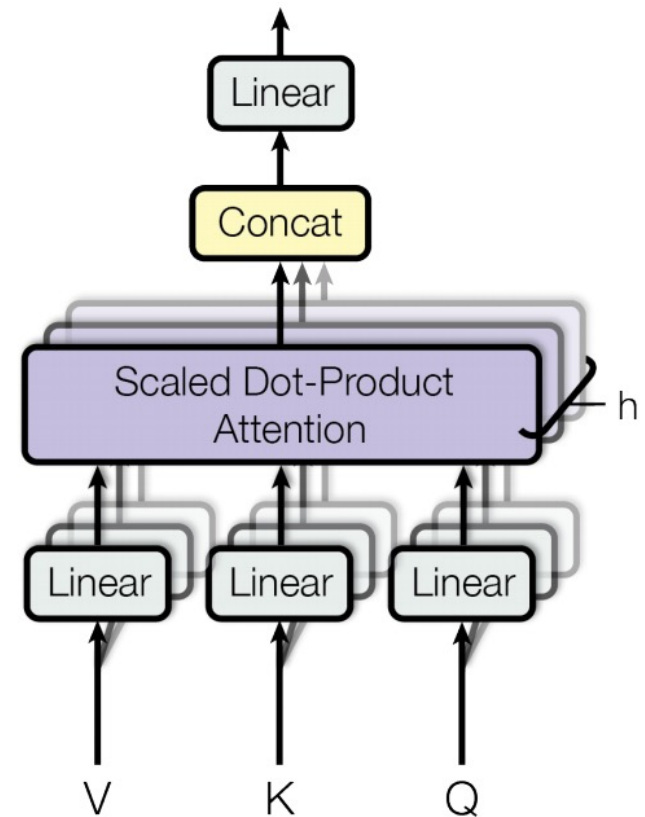
- To give transformer information about ordering of tokens, add function of position (based on sines and cosines) to every input



[Image source](#)

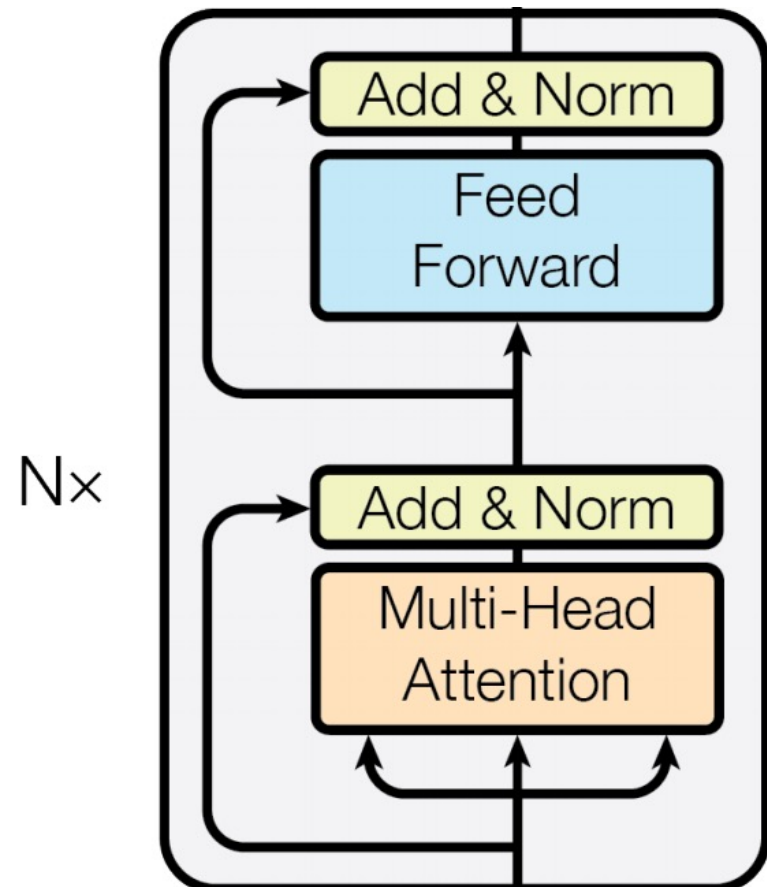
Multi-head attention

- Run h attention models in parallel on top of different linearly projected versions of Q, K, V ; concatenate and linearly project the results
- Intuition: enables model to attend to different kinds of information at different positions (see [visualization tool](#))

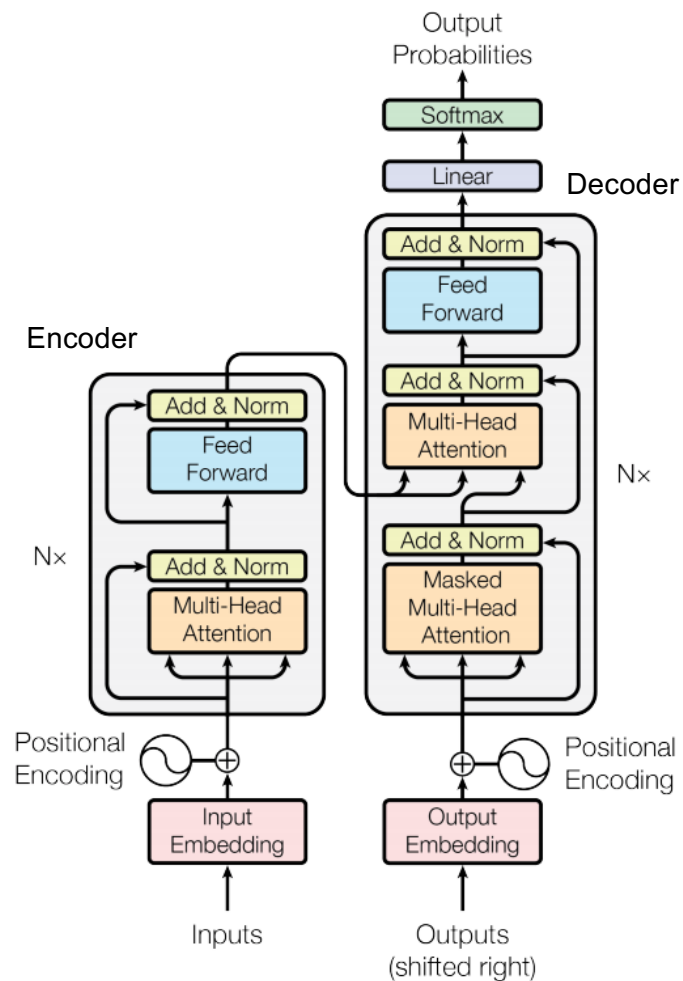


Transformer blocks

- A **Transformer** is a sequence of transformer blocks
 - Vaswani et al.: $N=12$ blocks, embedding dimension = 512, 6 attention heads
 - **Add & Norm**: residual connection followed by [layer normalization](#)
 - **Feedforward**: two linear layers with ReLUs in between, applied independently to each vector
- Attention is the only interaction between inputs!



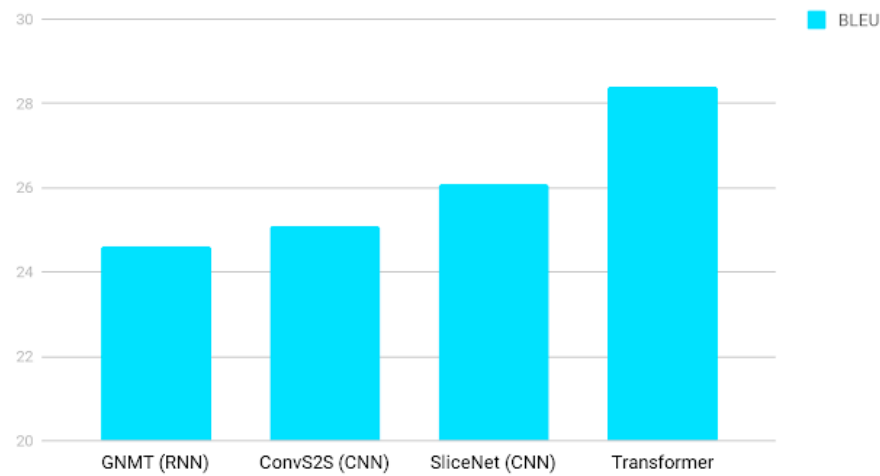
Transformer architecture: Zooming back out



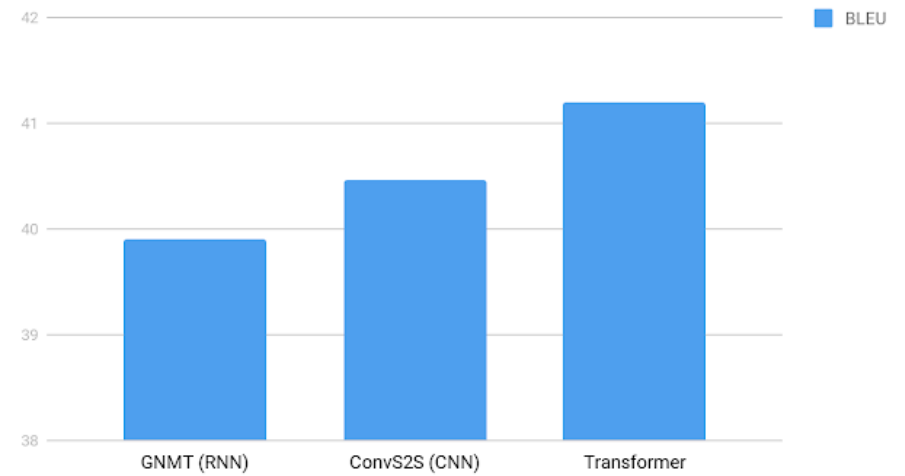
A. Vaswani et al., [Attention is all you need](#), NeurIPS 2017

Results

English German Translation quality



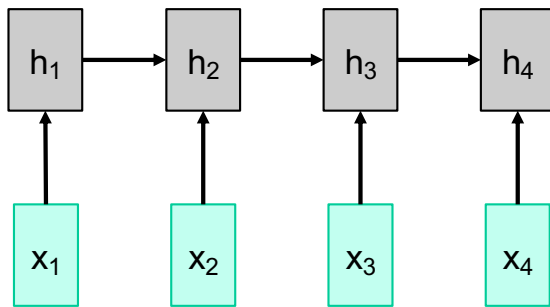
English French Translation Quality



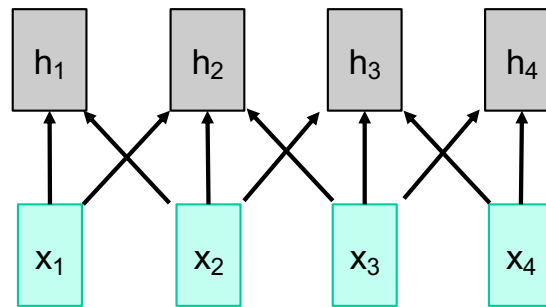
<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

Transformers: Pros and cons

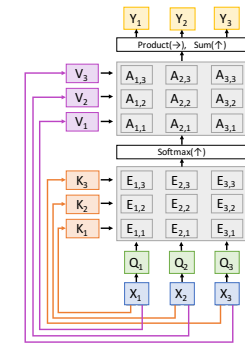
RNNs



1D convolutional networks



Transformers



Works on **ordered sequences**

- Pros: Not limited by fixed context size (in principle): After one RNN layer, h_T "sees" the whole sequence
- Con: Not parallelizable: need to compute hidden states sequentially
- Con: Hidden states have limited expressive capacity

Works on **multidimensional grids**

- Pro: Each output can be computed in parallel (at training time)
- Con: Need to stack many conv layers for outputs to "see" the whole sequence

• Works on **sets of vectors**

- Pro: Good at long sequences: after one self-attention layer, each output "sees" all inputs!
- Pro: Each output can be computed in parallel (at training time)
- Con: Memory-intensive: cost of attention operator is *quadratic* in input size

Making transformers more efficient

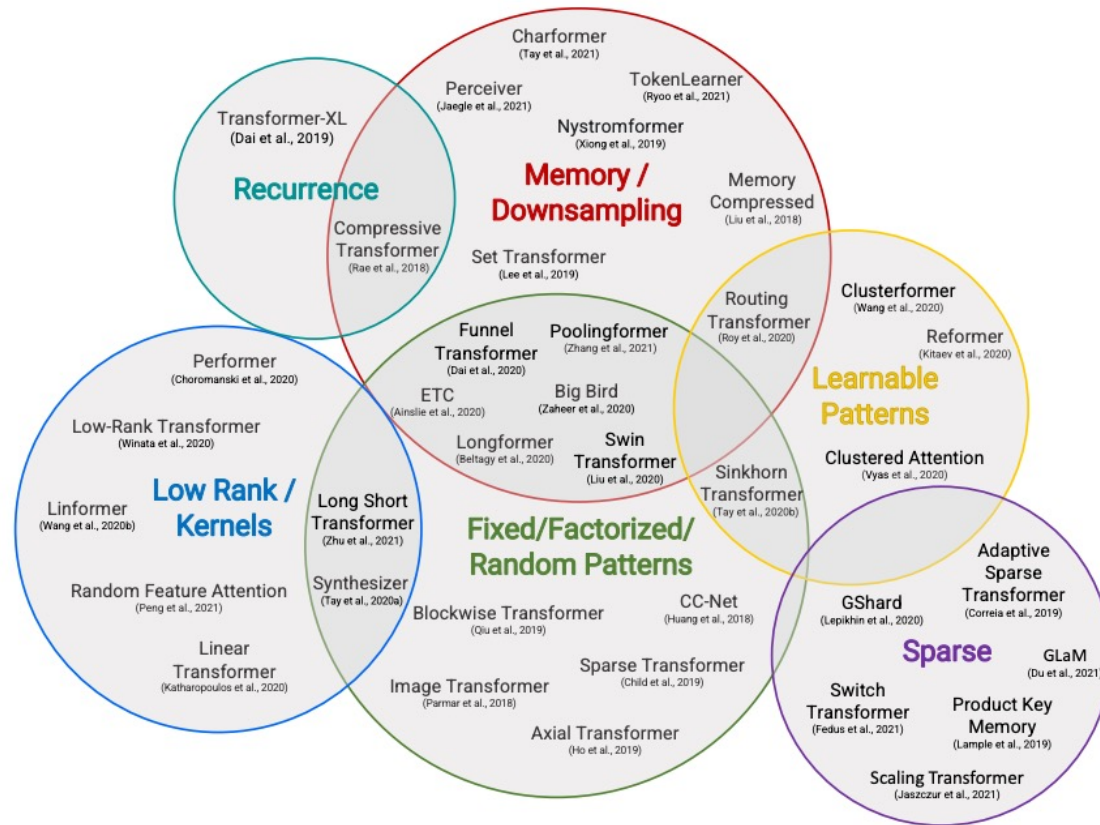


Figure 2: Taxonomy of Efficient Transformer Architectures.

Y. Tay et al. [Efficient transformers: A survey](#). arXiv 2022

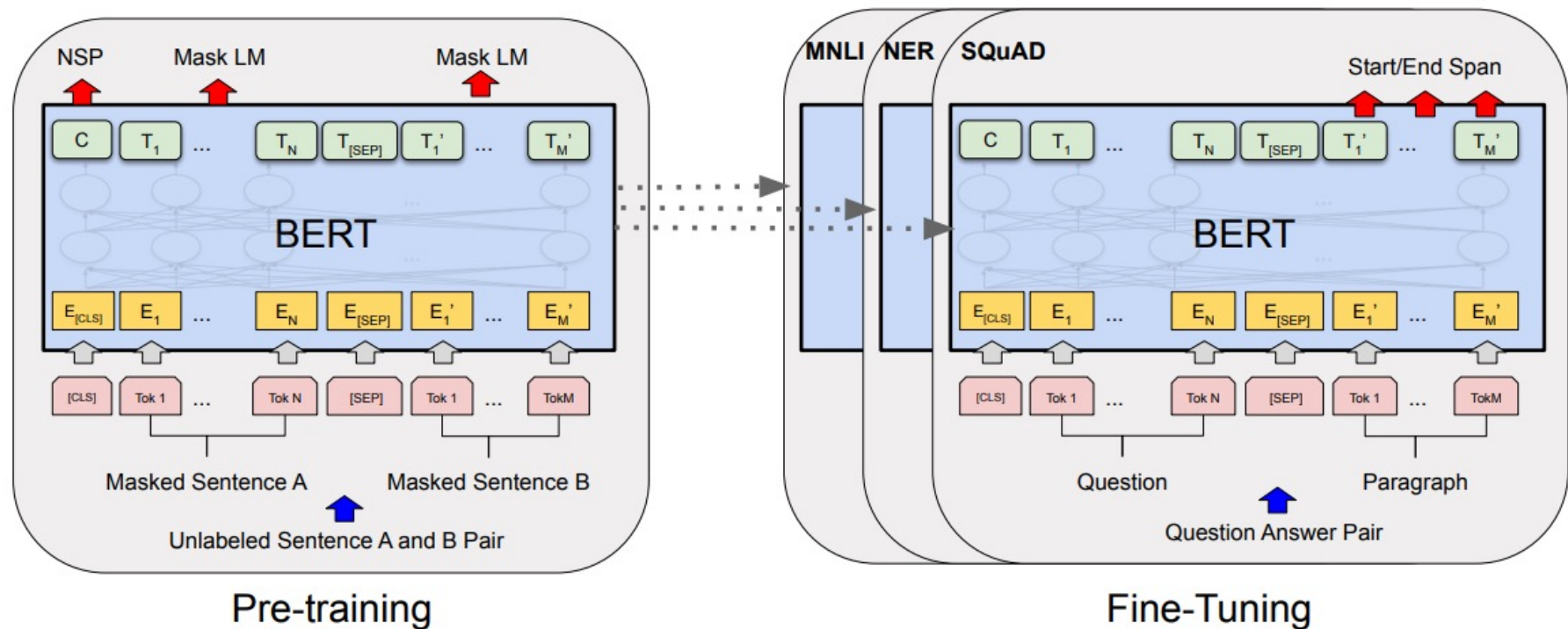
Outline

- Key-value-query attention
- Transformer attention models
- Other details of transformer architecture
- Large language models: the early years

Self-supervised language modeling with transformers

1. Download A LOT of text from the internet
2. Train a giant transformer using a suitable pretext task
3. Fine-tune the transformer on desired NLP task (optional)

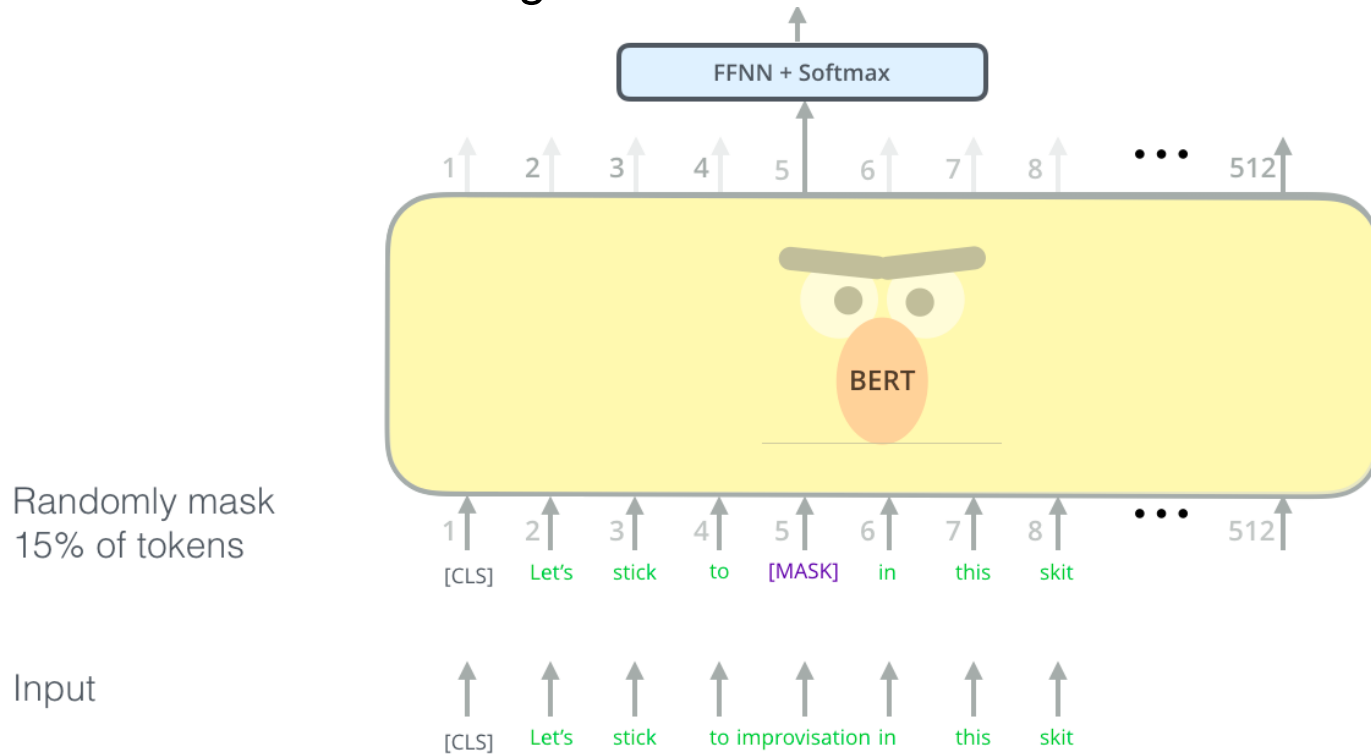
Bidirectional encoder representations from transformers (BERT)



J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#), EMNLP 2018

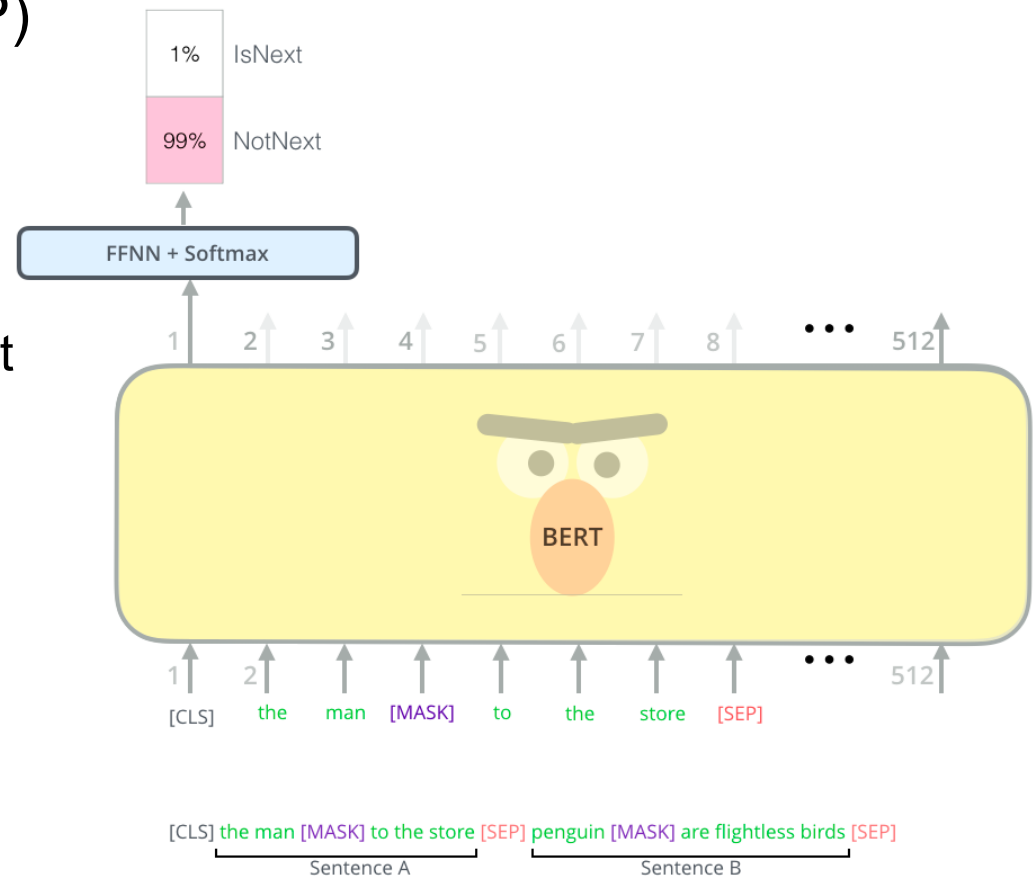
BERT: Pretext tasks

- Masked language model (MLM)
 - Randomly mask 15% of tokens in input sentences, goal is to reconstruct them using bidirectional context



BERT: Pretext tasks

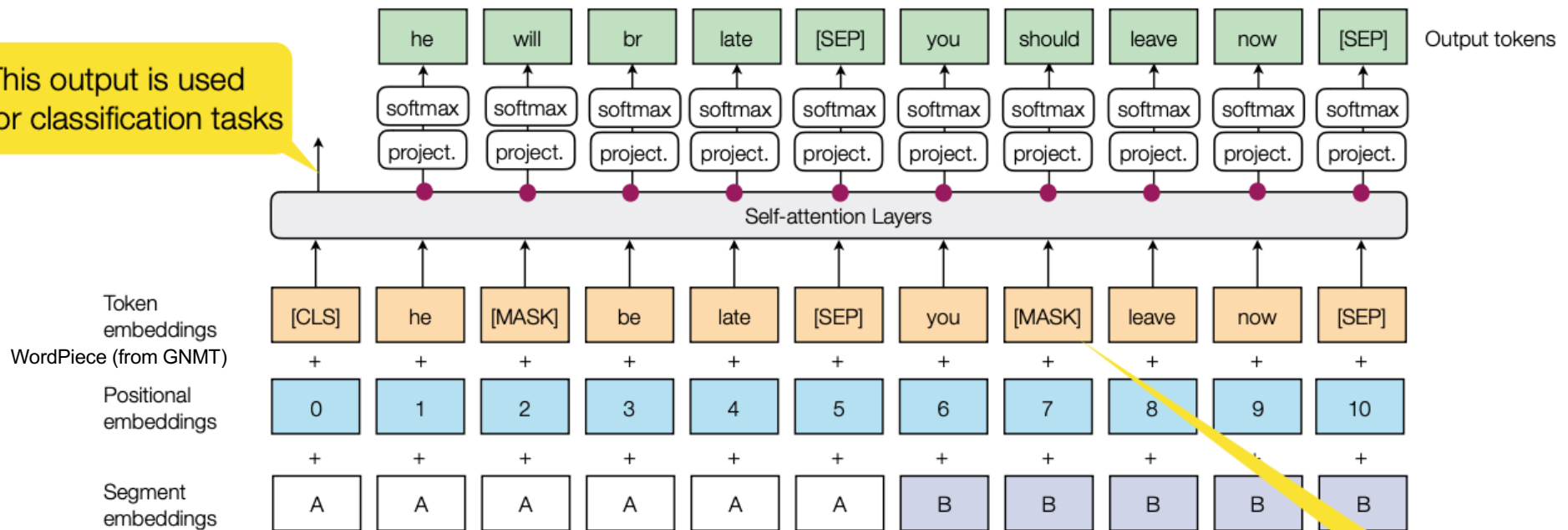
- Next sentence prediction (NSP)
 - Useful for Question Answering and Natural Language Inference tasks
 - In the training data, 50% of the time B is the actual sentence that follows A (labeled as IsNext), and 50% of the time it is a random sentence (labeled as NotNext).



[Image source](#)

BERT: More detailed view

This output is used for classification tasks



Trained on Wikipedia (2.5B words) + BookCorpus (800M words)

15% of tokens get masked

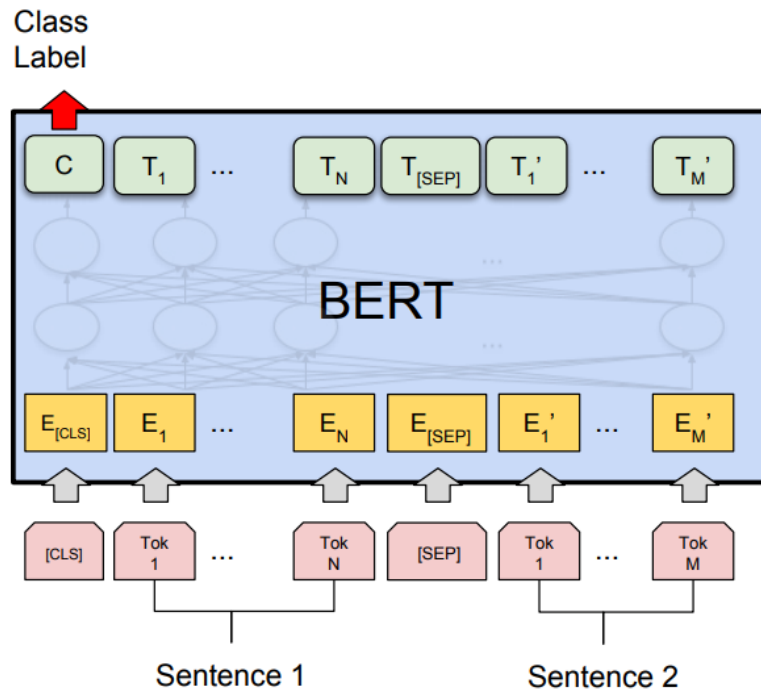
[Image source](#)

BERT: Evaluation

- General Language Understanding Evaluation (GLUE) benchmark (gluebenchmark.com)

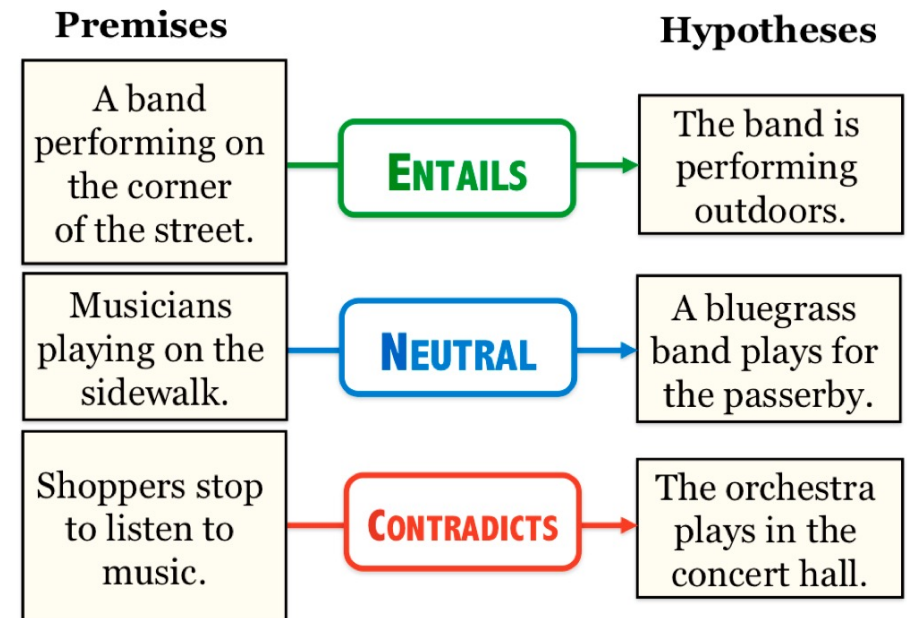
System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

BERT: Downstream tasks



(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG

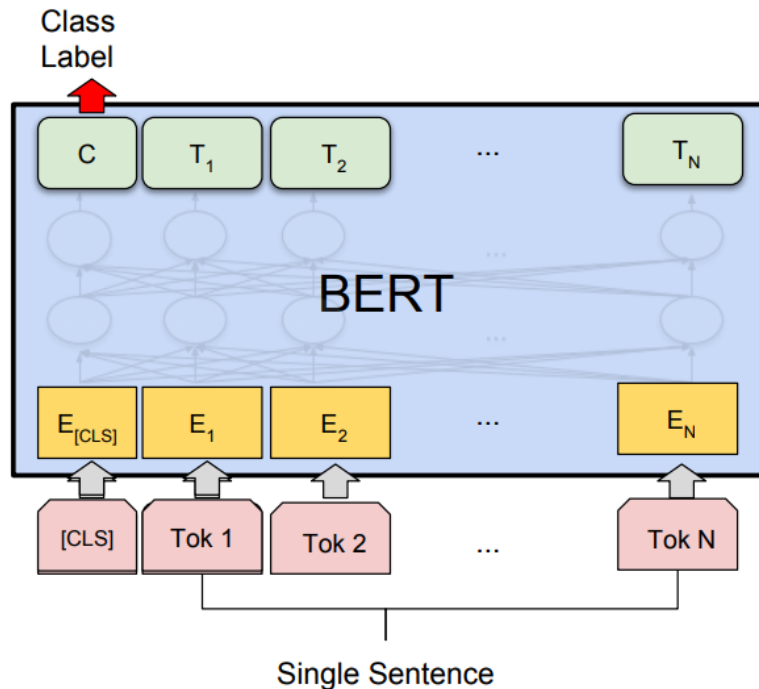
Textual entailment



Source: J. Hockenmaier

Entailment, textual equivalence and similarity

BERT: Downstream tasks



(b) Single Sentence Classification Tasks:
SST-2, CoLA

CoLa

Sentence: The wagon rumbled down the road.

Label: Acceptable

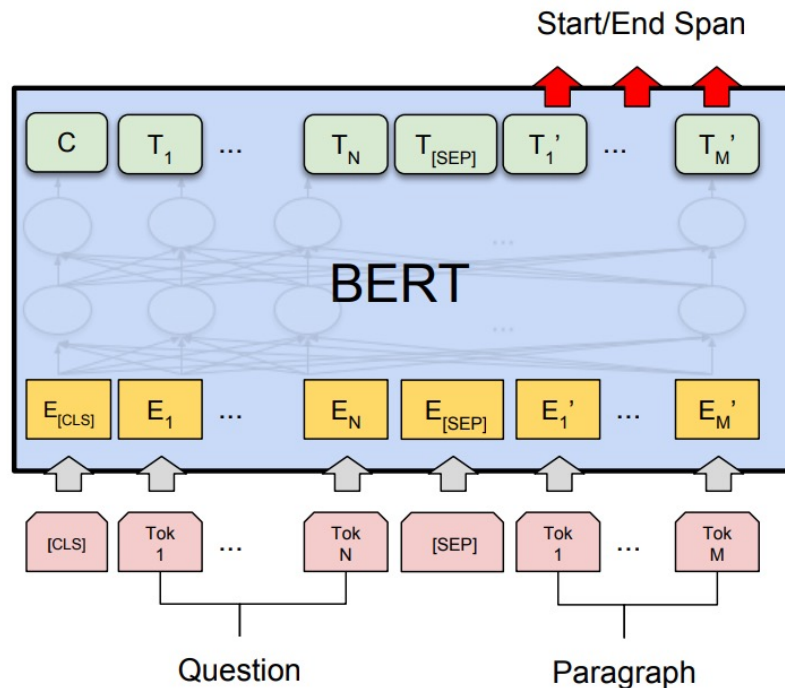
Sentence: The car honked down the road.

Label: Unacceptable

Sentiment classification, linguistic acceptability

[Image source](#)

BERT: Downstream tasks



(c) Question Answering Tasks:
SQuAD v1.1

Find span in paragraph that contains the answer

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **grau-pel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called “showers”.

What causes precipitation to fall?

gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?

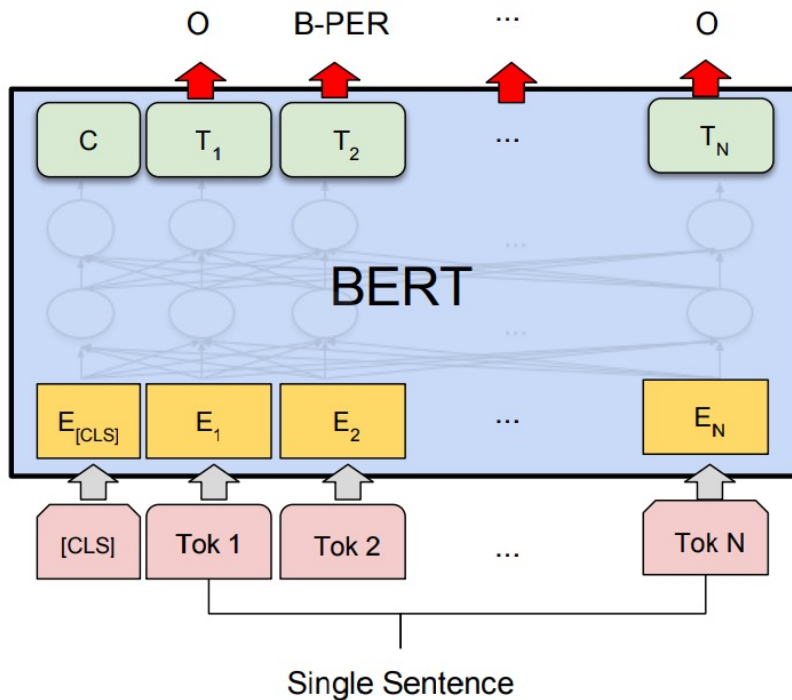
grau-pel

Where do water droplets collide with ice crystals to form precipitation?

within a cloud

Source: [SQuAD v1.1 paper](#)

BERT: Downstream tasks



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Named entity recognition

When **Sebastian Thrun** PERSON started at **Google** ORG in **2007** DATE, few people outside of the company took him seriously. "I can tell you very senior CEOs of major **American** NORP car companies would shake my hand and turn away because I wasn't worth talking to," said **Thrun** PERSON, now the co-founder and CEO of online higher education startup Udacity, in an interview with **Recode** ORG **earlier this week** DATE.

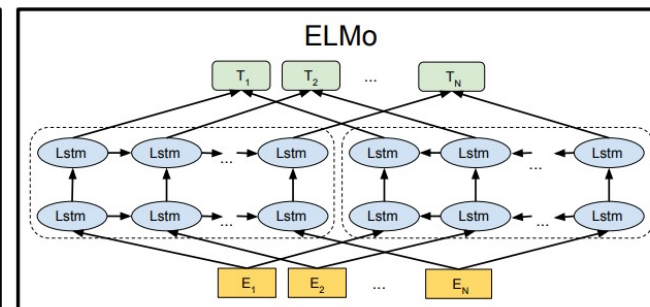
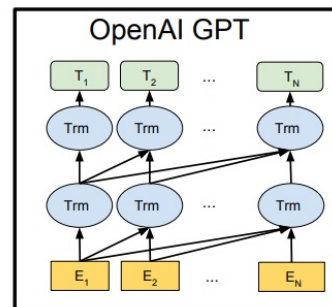
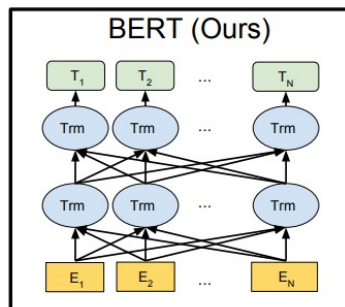
A little **less than a decade later** DATE, dozens of self-driving startups have cropped up while automakers around the world clamor, wallet in hand, to secure their place in the fast-moving world of fully automated transportation.

[Image source](#)

Other early language models

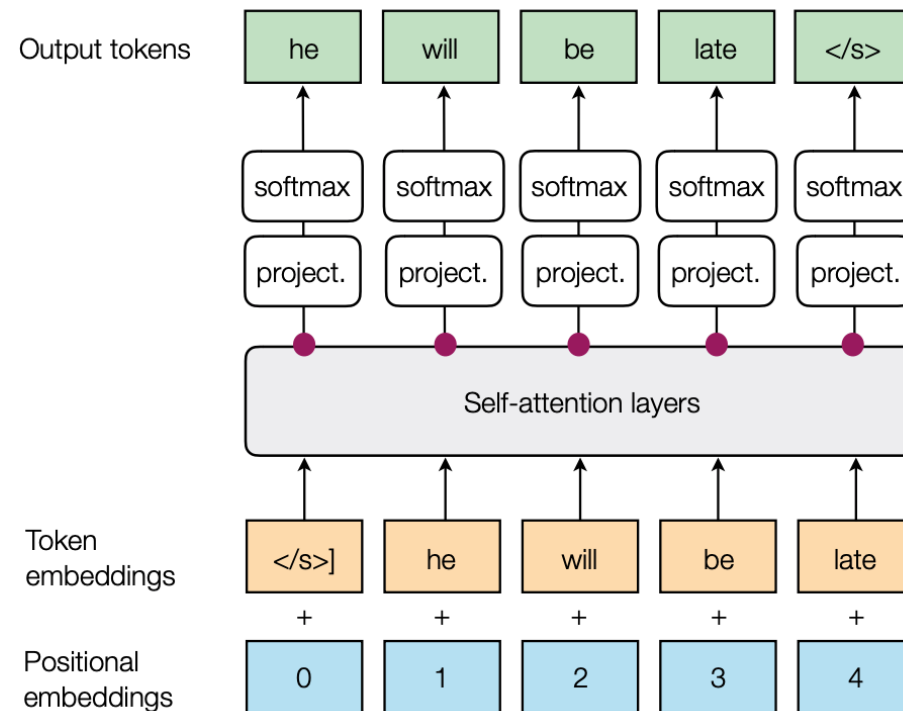
Alias	Model	Token	Tasks	Language
ULMfit	LSTM	word	Causal LM	English
ELMo	LSTM	word	Bidirectional LM	English
OpenAI GPT	Transformer	subword	Causal LM + Classification	English
BERT	Transformer	subword	Masked LM + Next sentence prediction	Multilingual
XLNet	Transformer	subword	Causal LM + Masked LM + Translation LM	Multilingual

[Image source](#)



OpenAI GPT (Generative Pre-Training)

- Pre-training task: next token prediction (causal language modeling)



A. Radford et al. [Improving language understanding by generative pre-training](#). 2018

[Image source](#)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)

Vaswani et al. [Attention is all you need](#). NeurIPS 2017 (Google)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)

Devlin et al. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#). EMNLP 2018 (Google)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)

Yang et al. [XLNet: Generalized Autoregressive Pretraining for Language Understanding](#). 2019 (Google, CMU)

Liu et al. [RoBERTa: A Robustly Optimized BERT Pretraining Approach](#). 2019 (FAIR, UW)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	

Radford et al. [Language models are unsupervised multitask learners](#). 2019 (OpenAI)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)

~\$430,000 on Amazon AWS

Shoeybi et al. [Megatron-LM: Training Multi-Billion Parameter Language Models using Model Parallelism](#). 2019 (NVIDIA)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU

Microsoft. [Turing-NLG: A 17-billion parameter language model by Microsoft](#). 2020

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12288	96	175B	694 GB	?

~\$4.6M, 355 GPU-years
([source](#))

Brown et al. [Language Models are Few-Shot Learners](#). NeurIPS 2020 (OpenAI)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12,288	96	175B	694 GB	?
GOPHER	80	16,384	128	280B	10.55 TB	4096x TPuv3 (38 days)

\$3,768,320 on Google Cloud (eval price)

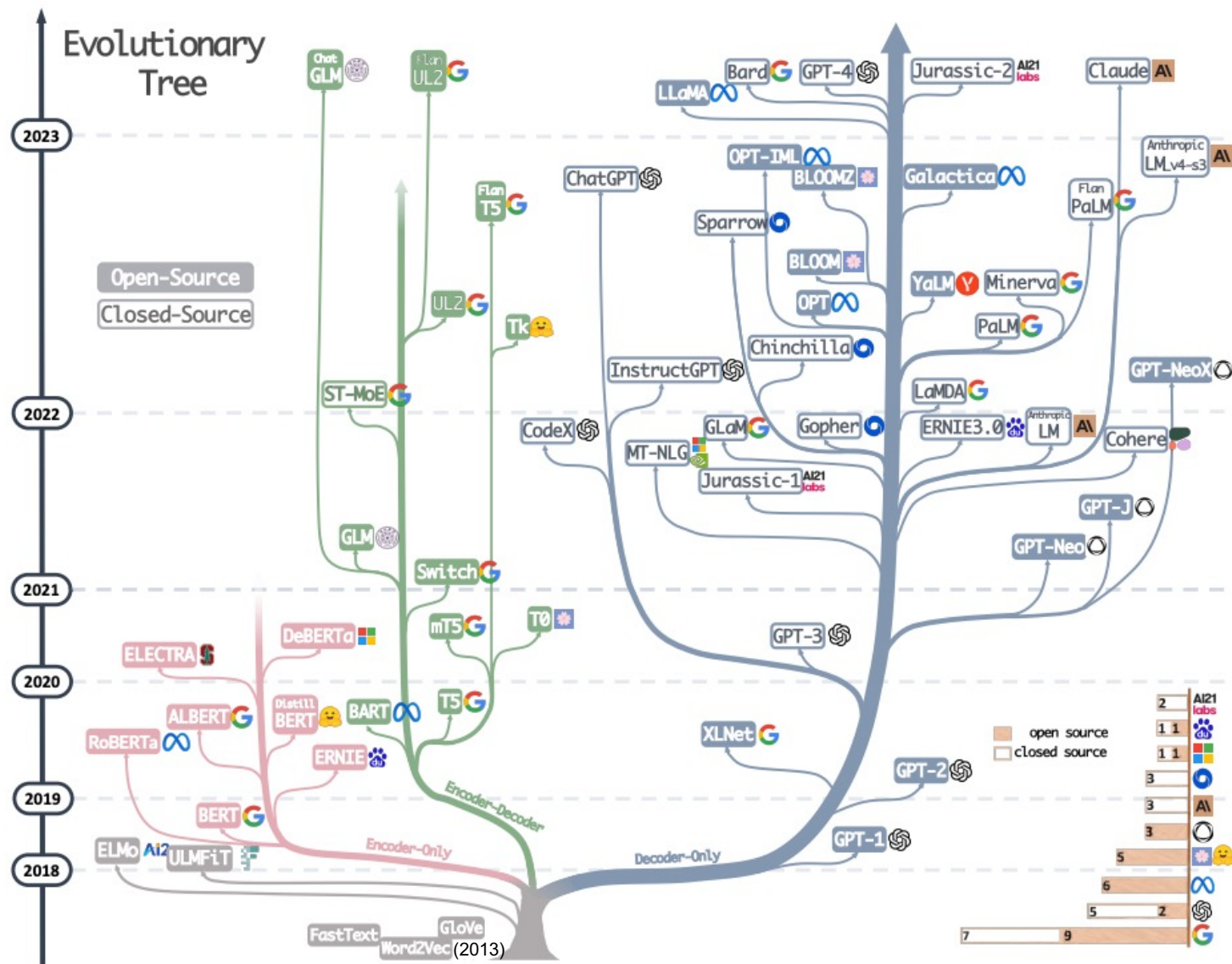
J. Rae et al. [Scaling Language Models: Methods, Analysis, & Insights from Training Gopher](#), arXiv 2021 (DeepMind)

Source: [J. Johnson](#)

Scaling up transformers

Model	Layers	Hidden dim.	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	4x TPU (4 days)
BERT-Large	24	1024	16	340M	13 GB	16x TPU (4 days)
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)
Turing-NLG	78	4256	28	17B	?	256x V100 GPU
GPT-3	96	12,288	96	175B	694 GB	?
GOPHER	80	16,384	128	280B	10.55 TB	4096x TPUv3 (38 days)
PaLM	118	18,432	48	540B	?	6144x TPUv4

A. Chowdhery et al. [PaLM: Scaling Language Modeling with Pathways](#). arXiv 2022 (Google)



[Source](#)